Bhoomi Annamdevara
24BCI0130

COMPILER DESIGN LAB
Lab Assignment - 06

Design of a Simple Arithmetic Expression Optimizer using Constant Folding

Aim:

To implement a program in C that performs Constant Folding on a simple arithmetic expression evaluator. The program reads an arithmetic expression containing constants, variables, and basic arithmetic operators (+, -, *, /), identifies and evaluates the constant sub-expressions at compile time, preserves the variables unchanged, and displays the optimized expression along with the value of the folded constant expression.

Algorithm:

1. Start the program.
2. Accept an assignment statement of the form lhs = rhs; from the user.
3. Split the statement at the '=' symbol into the left-hand side (lhs) and the right-hand side (rhs).
4. Remove the trailing semicolon, if present, from the rhs.
5. Tokenize the rhs into signed terms by splitting only at top-level '+' and '-' operators, carrying the sign forward with each term.
6. For each term, check whether it consists purely of digits combined with '*' or '/' (a constant term).
7. If the term is a constant term, evaluate it and add its signed value to a running constant sum.
8. If the term contains a variable, store it unchanged along with its sign.
9. After processing all terms, print the variable terms first, followed by the folded constant sum with its correct sign.
10. Display the value of the folded constant expression separately.
11. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX 200

char input[MAX];
char lhs[50];
char rhs[MAX];

char varTerms[50][50];
int varSign[50];
int varCount = 0;

int constSum = 0;

int isNumberTerm(char term[])
{
```

```
    int i = 0;
    if(term[i]=='\0')
        return 0;
    for(; term[i]!='\0'; i++)
    {
        if(!isdigit(term[i]) && term[i]!='*' && term[i]!='/')
            return 0;
    }
    return 1;
}

int evalConstTerm(char term[])
{
    /* term contains only digits, * and / , e.g. 22*26 or 10/2 */
    int result = 0;
    char numbuf[20];
    int ni = 0;
    char op = '*';
    int first = 1;

    for(int i = 0; i <= (int)strlen(term); i++)
    {
        if(isdigit(term[i]))
        {
            numbuf[ni++] = term[i];
        }
        else
        {
            numbuf[ni] = '\0';
            int val = atoi(numbuf);
            if(first)
            {
                result = val;
                first = 0;
            }
            else
            {
                if(op == '*')
                    result = result * val;
                else if(op == '/')
                    result = result / val;
            }
            op = term[i];
            ni = 0;
        }
    }
    return result;
}

int main()
{
    int sign[50];
    char terms[50][50];
    int termCount = 0;

    printf("Enter the Input String : ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    /* Split at '=' into lhs and rhs */
    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
        printf("Invalid statement, no '=' found\n");
        return 0;
```

```
}
int lhsLen = eq - input;
strncpy(lhs, input, lhsLen);
lhs[lhsLen] = '\0';
strcpy(rhs, eq + 1);

/* remove trailing ';' if present */
int rl = strlen(rhs);
if(rl > 0 && rhs[rl-1] == ';')
    rhs[rl-1] = '\0';

/* Tokenize rhs into signed terms, splitting only on top-level + and - */
int i = 0, j = 0;
char term[50];
int curSign = 1;
int len = strlen(rhs);

while(i <= len)
{
    if(rhs[i] == '+' || rhs[i] == '-' || rhs[i] == '\0')
    {
        term[j] = '\0';
        if(j > 0)
        {
            strcpy(terms[termCount], term);
            sign[termCount] = curSign;
            termCount++;
        }
        curSign = (rhs[i] == '-') ? -1 : 1;
        j = 0;
    }
    else
    {
        term[j++] = rhs[i];
    }
    i++;
}

/* Classify each term as constant or variable-containing */
for(int k = 0; k < termCount; k++)
{
    if(isNumberTerm(terms[k]))
    {
        int val = evalConstTerm(terms[k]);
        constSum += sign[k] * val;
    }
    else
    {
        varSign[varCount] = sign[k];
        strcpy(varTerms[varCount], terms[k]);
        varCount++;
    }
}

/* Build and print optimized expression */
printf("%s=", lhs);
for(int k = 0; k < varCount; k++)
{
    if(varSign[k] == -1)
        printf("-%s", varTerms[k]);
    else
    {
        if(k == 0)
            printf("%s", varTerms[k]);
        else
```

```
                printf("+%s", varTerms[k]);
        }
    }

    if(constSum >= 0)
    {
        if(varCount > 0)
            printf("+%d\n", constSum);
        else
            printf("%d\n", constSum);
    }
    else
    {
        printf("%d\n", constSum);
    }

    printf("The Value of Constant Expression is : %d\n", constSum);

    return 0;
}
```

Input/Output Screenshots:



main.c

Run ▶

```c
    int result = 0;
    char numbuf[20];
    int ni = 0;
    char op = '*';
    int first = 1;

    for(int i = 0; i <= (int)strlen(term); i++)
    {
        if(isdigit(term[i]))
        {
            numbuf[ni++] = term[i];
        }
        else
        {
            numbuf[ni] = '\0';
            int val = atoi(numbuf);
            if(first)
            {
                result = val;
                first = 0;
            }
            else
            {
                if(op == '*')
                    result = result * val;
                else if(op == '/')
                    result = result / val;
            }
            op = term[i];
            ni = 0;
        }
    }
    return result;
}
```

main.c

Run ▶

```c

int main()
{
    int sign[50];
    char terms[50][50];
    int termCount = 0;

    printf("Enter the Input String : ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    /* Split at '=' into lhs and rhs */
    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
        printf("Invalid statement, no '=' found\n");
        return 0;
    }
    int lhsLen = eq - input;
    strncpy(lhs, input, lhsLen);
    lhs[lhsLen] = '\0';
    strcpy(rhs, eq + 1);

    /* remove trailing ';' if present */
    int rl = strlen(rhs);
    if(rl > 0 && rhs[rl-1] == ';')
        rhs[rl-1] = '\0';

    /* Tokenize rhs into signed terms, splitting only on top-level + and
- */
    int i = 0, j = 0;
    char term[50];
    int curSign = 1;
    int len = strlen(rhs);

```

main.c

Run ▶

```
    int curSign = 1;
    int len = strlen(rhs);

    while(i <= len)
    {
        if(rhs[i] == '+' || rhs[i] == '-' || rhs[i] == '\0')
        {
            term[j] = '\0';
            if(j > 0)
            {
                strcpy(terms[termCount], term);
                sign[termCount] = curSign;
                termCount++;
            }
            curSign = (rhs[i] == '-') ? -1 : 1;
            j = 0;
        }
        else
        {
            term[j++] = rhs[i];
        }
        i++;
    }

    /* Classify each term as constant or variable-containing */
    for(int k = 0; k < termCount; k++)
    {
        if(isNumberTerm(terms[k]))
        {
            int val = evalConstTerm(terms[k]);
            constSum += sign[k] * val;
        }
        else
        {
            varSign[varCount] = sign[k];
```

main.c

Run ▶

```
    /* Classify each term as constant or variable-containing */
    for(int k = 0; k < termCount; k++)
    {
        if(isNumberTerm(terms[k]))
        {
            int val = evalConstTerm(terms[k]);
            constSum += sign[k] * val;
        }
        else
        {
            varSign[varCount] = sign[k];
            strcpy(varTerms[varCount], terms[k]);
            varCount++;
        }
    }

    /* Build and print optimized expression */
    printf("%s=", lhs);
    for(int k = 0; k < varCount; k++)
    {
        if(varSign[k] == -1)
            printf("-%s", varTerms[k]);
        else
        {
            if(k == 0)
                printf("%s", varTerms[k]);
            else
                printf("+%s", varTerms[k]);
        }
    }

    if(constSum >= 0)
    {
        if(varCount > 0)
            printf("+%d\n", constSum);
```

main.c

Run ▶

```
        }
    }

    /* Build and print optimized expression */
    printf("%s=", lhs);
    for(int k = 0; k < varCount; k++)
    {
        if(varSign[k] == -1)
            printf("-%s", varTerms[k]);
        else
        {
            if(k == 0)
                printf("%s", varTerms[k]);
            else
                printf("+%s", varTerms[k]);
        }
    }

    if(constSum >= 0)
    {
        if(varCount > 0)
            printf("+%d\n", constSum);
        else
            printf("%d\n", constSum);
    }
    else
    {
        printf("%d\n", constSum);
    }

    printf("The Value of Constant Expression is : %d\n", constSum);

    return 0;
```

Output

```
Enter the Input String : a=b+22*26+c-d+10
a=b+c-d+582
The Value of Constant Expression is : 582


=== Code Execution Successful ===
```

Output

```
Enter the Input String : a=4*5+a-b+10+5+c;
a=a-b+c+35
The Value of Constant Expression is : 35


=== Code Execution Successful ===
```

Output

```
Enter the Input String : a=Isla+10/2+22-9;
a=Isla+18
The Value of Constant Expression is : 18


=== Code Execution Successful ===
```

Output

```
Enter the Input String : a=b+10+c;
a=b+c+10
The Value of Constant Expression is : 10


=== Code Execution Successful ===
```

Output

```
Enter the Input String : result=24+bhoomi+0130-annamdevara+10;
result=bhoomi-annamdevara+164
The Value of Constant Expression is : 164


=== Code Execution Successful ===
```

Result:

Thus, the program to perform Constant Folding as a compiler optimization technique was successfully developed. It reads an arithmetic expression, identifies and evaluates the constant sub-expressions, preserves the variables unchanged, and displays the optimized expression along with the value of the folded constant expression.

Identify the Common Sub-expression in a Given Statement

Aim:

To implement a program in C that identifies common sub-expressions within a single assignment statement, as part of a compiler optimization module aimed at reducing redundant calculations and improving code efficiency.

Algorithm:

1. Start the program.
2. Accept one assignment statement of the form lhs = rhs; from the user.
3. Split the statement at the '=' symbol and extract the rhs.
4. Remove the trailing semicolon, if present, from the rhs.
5. Scan the rhs left to right and split it into individual operands at every top-level '+' or '*' operator, recording the operator between each pair of operands.
6. Form a binary sub-expression (operand <operator> operand) between every pair of consecutive operands and store it in an expression list.
7. Display all the sub-expressions formed, numbered in order.
8. Compare every pair of sub-expressions in the list for an exact match.
9. If two or more sub-expressions match, group and report them together as a common expression along with their expression numbers.
10. If no sub-expressions match, report that there is no common expression in the given statement.
11. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 200

char input[MAX];
char rhs[MAX];
char expr[100][50];
int exprCount = 0;

int main()
{
    printf("Enter the Input String : ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *eq = strchr(input, '=');
    if(eq == NULL)
    {
        printf("Invalid statement, no '=' found\n");
        return 0;
    }
    strcpy(rhs, eq + 1);

    int rl = strlen(rhs);
```

```
if(rl > 0 && rhs[rl-1] == ';')
    rhs[rl-1] = '\0';

/* Build binary sub-expressions: operand <op> operand,
   formed between every pair of consecutive operators
   (+ or *) as we scan left to right. */
char operands[100][50];
char ops[100];
int opCount = 0;

int i = 0, j = 0;
char token[50];
int len = strlen(rhs);

while(i <= len)
{
    if(rhs[i] == '+' || rhs[i] == '*' || rhs[i] == '\0')
    {
        token[j] = '\0';
        strcpy(operands[opCount], token);
        if(rhs[i] != '\0')
            ops[opCount] = rhs[i];
        opCount++;
        j = 0;
    }
    else
    {
        token[j++] = rhs[i];
    }
    i++;
}
/* opCount = number of operands; ops has opCount-1 valid operators */

/* Form a sub-expression for each consecutive operand pair,
   using the operator between them */
for(int k = 0; k < opCount - 1; k++)
{
    sprintf(expr[exprCount], "%s%c%s", operands[k], ops[k], operands[k+1]);
    exprCount++;
}

printf("\nThe Expressions available in the given statements are\n");
for(int k = 0; k < exprCount; k++)
    printf("Expression %d : %s\n", k+1, expr[k]);

printf("\nThe Common Expression Available in given statement are\n");

int visited[100] = {0};
int found = 0;

for(int a = 0; a < exprCount; a++)
{
    if(visited[a]) continue;
    int first = 1;
    for(int b = a+1; b < exprCount; b++)
    {
        if(strcmp(expr[a], expr[b]) == 0)
        {
            if(first)
            {
                printf("Expression%d", a+1);
                first = 0;
                found = 1;
            }
            printf(" & Expression%d", b+1);
```

```
                visited[b] = 1;
            }
        }
        if(!first)
            printf(" --> %s\n", expr[a]);
    }

    if(!found)
        printf("There is no Common Expression in given statement\n");

    return 0;
}
```

Input/Output Screenshots:



main.c

Run ▶

```
    char operands[100][50];
    char ops[100];
    int opCount = 0;

    int i = 0, j = 0;
    char token[50];
    int len = strlen(rhs);

    while(i <= len)
    {
        if(rhs[i] == '+' || rhs[i] == '*' || rhs[i] == '\0')
        {
            token[j] = '\0';
            strcpy(operands[opCount], token);
            if(rhs[i] != '\0')
                ops[opCount] = rhs[i];
            opCount++;
            j = 0;
        }
        else
        {
            token[j++] = rhs[i];
        }
        i++;
    }
    /* opCount = number of operands; ops has opCount-1 valid operators */

    /* Form a sub-expression for each consecutive operand pair,
       using the operator between them */
    for(int k = 0; k < opCount - 1; k++)
    {
        sprintf(expr[exprCount], "%s%c%s", operands[k], ops[k],
operands[k+1]);
        exprCount++;
    }
```

main.c

Run ▶

```
        exprCount++;
    }

    printf("\nThe Expressions available in the given statements are\n");
    for(int k = 0; k < exprCount; k++)
        printf("Expression %d : %s\n", k+1, expr[k]);

    printf("\nThe Common Expression Available in given statement are\n");

    int visited[100] = {0};
    int found = 0;

    for(int a = 0; a < exprCount; a++)
    {
        if(visited[a]) continue;
        int first = 1;
        for(int b = a+1; b < exprCount; b++)
        {
            if(strcmp(expr[a], expr[b]) == 0)
            {
                if(first)
                {
                    printf("Expression%d", a+1);
                    first = 0;
                    found = 1;
                }
                printf(" & Expression%d", b+1);
                visited[b] = 1;
            }
        }
        if(!first)
            printf(" --> %s\n", expr[a]);
    }

```

main.c

Run ▶

```
        printf("Expression %d : %s\n", k+1, expr[k]);

    printf("\nThe Common Expression Available in given statement are\n");

    int visited[100] = {0};
    int found = 0;

    for(int a = 0; a < exprCount; a++)
    {
        if(visited[a]) continue;
        int first = 1;
        for(int b = a+1; b < exprCount; b++)
        {
            if(strcmp(expr[a], expr[b]) == 0)
            {
                if(first)
                {
                    printf("Expression%d", a+1);
                    first = 0;
                    found = 1;
                }
                printf(" & Expression%d", b+1);
                visited[b] = 1;
            }
        }
        if(!first)
            printf(" --> %s\n", expr[a]);
    }

    if(!found)
        printf("There is no Common Expression in given statement\n");

    return 0;
}
```

Output

```
Enter the Input String : a=b+c*d+f+c*d+b+c;

The Expressions available in the given statements are
Expression 1 : b+c
Expression 2 : c*d
Expression 3 : d+f
Expression 4 : f+c
Expression 5 : c*d
Expression 6 : d+b
Expression 7 : b+c

The Common Expression Available in given statement are
Expression1 & Expression7 --> b+c
Expression2 & Expression5 --> c*d
```

Output

```
Enter the Input String : a=b+c*d;

The Expressions available in the given statements are
Expression 1 : b+c
Expression 2 : c*d

The Common Expression Available in given statement are
There is no Common Expression in given statement


=== Code Execution Successful ===
```

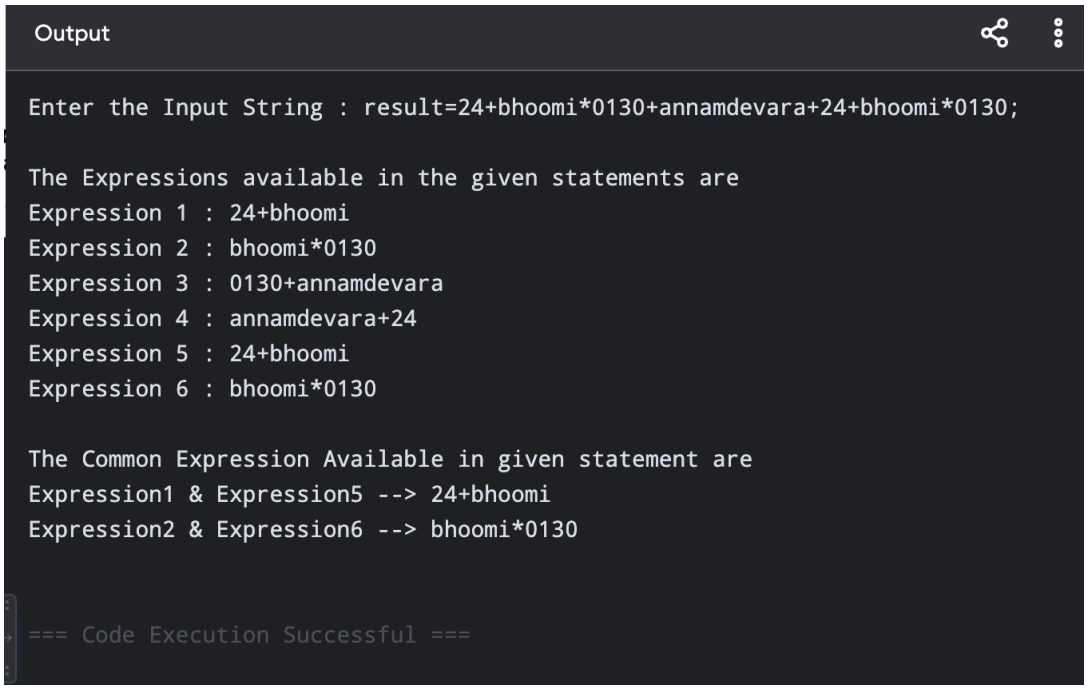
Output

```
Enter the Input String : Lab6=Compiler*Isla+Design+Compiler*Isla+Test;

The Expressions available in the given statements are
Expression 1 : Compiler*Isla
Expression 2 : Isla+Design
Expression 3 : Design+Compiler
Expression 4 : Compiler*Isla
Expression 5 : Isla+Test

The Common Expression Available in given statement are
Expression1 & Expression4 --> Compiler*Isla


=== Code Execution Successful ===
```



Result:

Thus, the program to identify common sub-expressions in a given assignment statement was successfully developed. It correctly parses the statement, forms the constituent sub-expressions, and identifies all sub-expressions that occur more than once within the statement.